Technical Assignment: General Nephrology EMR : Patient Management & AI Clinical Assistant

1. Objective

Develop a small, functional Electronic Medical Record (EMR) application for a general nephrology clinic.

The application should enable nephrologists to manage patient information, document clinical encounters, review laboratory results, monitor kidney function trends, and generate AI-assisted clinical summaries.

The objective is to evaluate your full-stack development capabilities, understanding of AI engineering, problem-solving skills, and ability to build practical software applications.

2. Technology Requirements

Frontend

- Angular
- TypeScript, HTML, and CSS.
- Use a charting library for laboratory trend visualization.

Backend

- Implement REST APIs for frontend-backend communication.
- .NET is preferred; however, candidates are free to choose their preferred backend technology, programming language, and libraries (e.g., Python, Node.js, or others).
- Focus on clean API design, proper request/response handling, and backend integration with the frontend.

Database

- Candidates may use any suitable SQL-based database (e.g., SQL Server, PostgreSQL, or SQLite).

- Alternatively, candidates may use locally stored JSON files or a predefined static dataset to store and retrieve patient information.
- The choice of data storage is flexible; the focus should be on effective data handling and integration with the application.

AI Integration

- Integrate an LLM using an available API such as OpenAI, Gemini, Claude, or an equivalent model. (optional if paid, use free/open source providers)
- Free-tier APIs or a documented mock implementation are acceptable where API access is restricted.

AI Coding Tools

- The use of AI coding assistants and agents is encouraged.
- You are free to choose whichever tools and development workflow you find most effective for building the application.
- Examples include GitHub Copilot, Cursor, Claude Code, Gemini CLI, or other coding assistants. (you can choose the open source / free tier models of your choice)
- We encourage effective use of these tools while maintaining an understanding of the generated code and implementation decisions.

3. Functional Requirements

A. Patient Management

Build a patient management module that supports:

- Patient listing with Patient ID, name, age, gender, and primary diagnosis.
- Search patients by name or patient ID.
- Patient demographics and medical history.
- Comorbidities and established diagnoses.
- Current medications and allergies.
- Previous clinical encounters.

B. Clinical Encounter Management

Implement an outpatient clinical encounter module.

Users should be able to:

- Create a new patient encounter.
- Record visit date and encounter details.
- Capture patient vitals such as BP, heart rate, temperature, and weight.
- Record chief complaints, symptoms, and clinical examination notes.
- Record clinical assessment and diagnoses.
- Document the treatment/follow-up plan.
- Save and retrieve previous encounters.

C. Laboratory Results & Kidney Function Monitoring

Implement a laboratory results module with the following parameters:

- Serum Creatinine.
- eGFR.
- Blood Urea Nitrogen (BUN).
- Potassium.
- Hemoglobin.
- Calcium.
- Phosphorus.
- Urine Albumin-Creatinine Ratio (UACR).

Users should be able to:

- Add laboratory results with dates and units.
- View historical laboratory results.
- Visualize Creatinine and eGFR trends using charts.
- Highlight abnormal laboratory values using supplied reference ranges.
- Display the established CKD stage or classify eGFR using the supplied classification criteria.

D. AI-Powered Clinical Assistant

Implement an AI-powered feature that generates a clinical summary for a selected patient.

The AI assistant should:

1. Summarize the patient's nephrology history.
2. Summarize relevant medical conditions and comorbidities.
3. Identify trends in Creatinine and eGFR based on historical results.
4. Highlight abnormal laboratory findings based on supplied reference ranges.
5. Identify missing or inconsistent clinical information.
6. Generate follow-up considerations for clinician review.

Technical expectations:

- Integrate an LLM through a backend service.
- Prepare relevant patient context before calling the LLM.
- Handle missing information without fabricating patient details.
- Implement error handling for invalid LLM responses and API failures.
- Clearly distinguish AI-generated information from original patient records.
- Include an appropriate clinical review disclaimer.

AI-generated content must not independently diagnose patients or prescribe treatment.

4. Engineering Expectations

- Use reusable Angular components.
- Implement clean and well-structured backend APIs.
- Maintain appropriate separation of concerns.
- Implement basic error handling and input validation.
- Maintain meaningful database relationships.
- Include basic testing for important functionality.
- Provide setup and execution instructions.

5. Submission Requirements

Submit:

1. Complete source code in a Git repository.
2. README with setup and execution instructions.
3. Database schema and synthetic patient data.
4. API documentation or endpoint descriptions.

5. A brief explanation of application architecture and AI integration.
6. Screenshots or a short screen recording demonstrating the application.

Use synthetic patient information only. Do not use real patient records or protected health information.

6. Additional Notes

- You are encouraged to use AI coding assistants and agents throughout the development process.
- You may choose the backend framework and database that best suit your implementation.
- .NET experience is an advantage but is not a mandatory requirement.
- Focus on delivering a functional application with a clean user experience and reliable backend integration.
- You should be able to explain your architecture, implementation choices, and AI integration during the technical discussion.